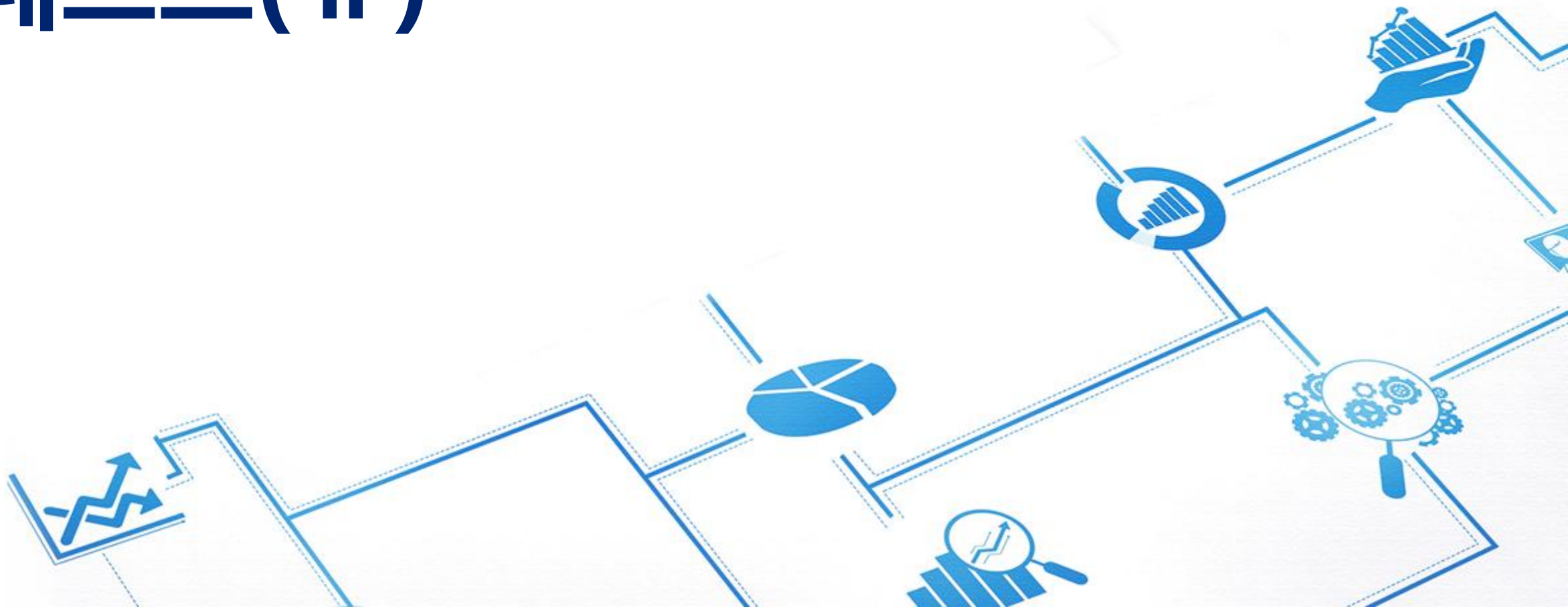


테스트(Ⅱ)



학습개요

본 주차의 학습목표와 학습내용을 확인하세요.

학습목표

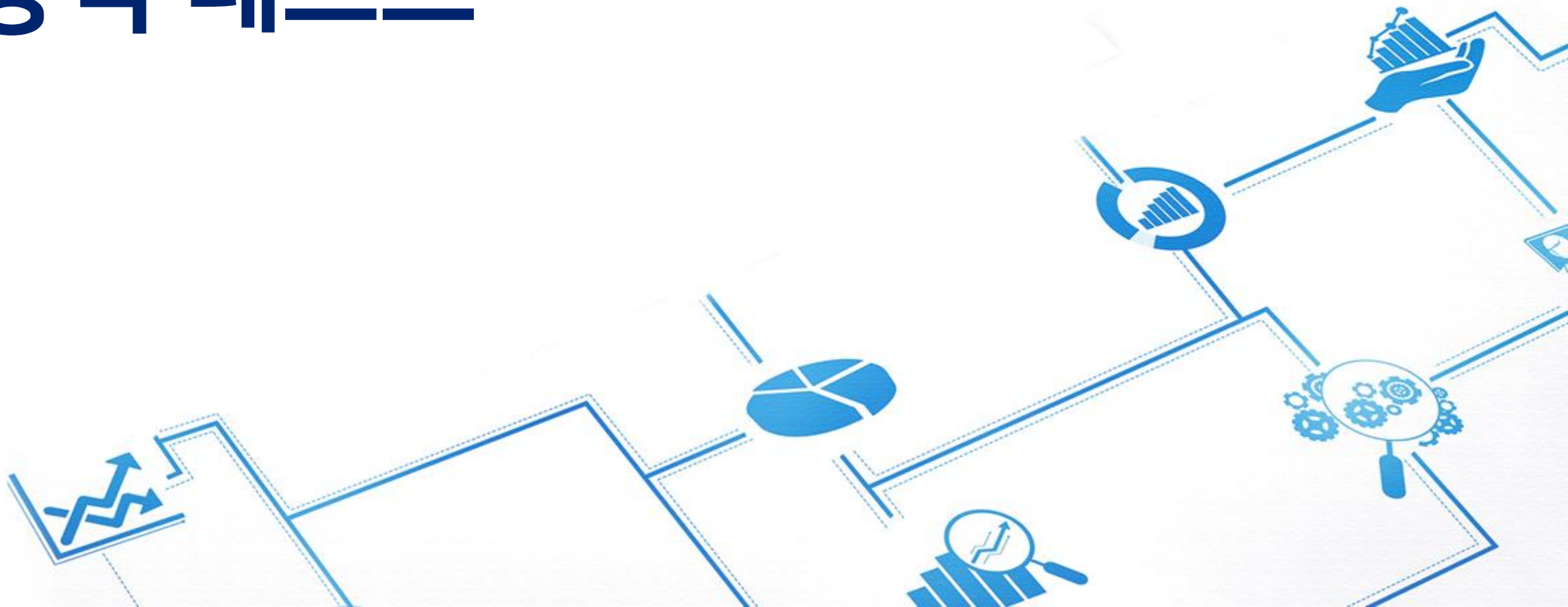
- ☑ 소프트웨어 테스트 프로세스에 대해 알아본다.
- ☑ 시각에 따른 테스트 기법에 대해 살펴본다.
- ☑ 목적에 따른 기법에 대해 살펴본다.
- ☑ 프로그램 실행 여부에 따른 테스트 기법에 대해 살펴본다.
- ☑ 개발 단계에 따른 테스트 기법에 대해 살펴본다.

학습내용

- 1 테스트의 이해, 테스트 분류
- 2 정적 테스트
- 3 동적 테스트
- 4 소프트웨어 개발 단계에 따른 테스트



동적 테스트



명세 기반 테스트

- 명세 기반 테스트(블랙박스 테스트)
 - 의사는 몸속을 직접 열어 확인하는 것이 아니라 다양한 의료 장비를 통해 병의 원인을 찾음
 - 입력 값에 대한 예상 출력 값을 정해놓고 그대로 결과가 나오는지 체크
 - 프로그램 내부의 구조나 알고리즘을 보지 않고, 요구 분석 명세서나 설계 사양서에서 테스트 케이스를 추출하여 테스트
 - 기능을 어떻게 수행하는가 보다는 사용자가 원하는 기능을 수행하는가 테스트



그림 9-14 블랙박스 테스트 비유: 청진기로 환자 진찰하기

명세 기반 테스트

- 선택스 기법
 - 가장 단순한 방법으로, 문법에 기반을 둔 테스트
 - 문법을 정해놓고 적합/부적합 입력 값에 따른 예상 결과가 제대로 나오는지 테스트

표 9-3 ID와 비밀번호의 적합 기준

	적합	부적합
ID	알파벳과 숫자(4~8자)	특수 기호 사용, 4자 미만, 9자 이상
비밀번호	알파벳·숫자·특수문자로 구성, 7글자 이상, 특수문자는 반드시 포함	7자 미만, 특수문자 미사용

표 9-4 적합/부적합 입력 값에 대한 예상 처리 결과

번호	적합/부적합	입력 값(ID/비번)	예상 처리 결과
1	적합	sogong/abcd#78	정상 등록
2	부적합	so*gong/abcd#78	에러 메시지: 등록 불가(ID-특수문자 사용)
3	부적합	sogong/abcdefg	에러 메시지: 등록 불가(비밀번호-특수문자 미사용)
4	부적합	gong/abcd	에러 메시지: 등록 불가(ID, 비밀번호-길이 부적합)

명세 기반 테스트

- 동등 분할 기법

- 각 영역에 해당하는 입력 값을 넣고 예상되는 출력 값이 나오는지 실제 값과 비교
- 단순하고 이해하기 쉬우며 사용자가 작성 가능하다는 장점

표 9-5 평가 점수에 따른 상여금

평가 점수	상여금
90~100점	600만 원
80~89점	400만 원
70~79점	200만 원
0~69점	0원

표 9-6 동등 분할 기법의 예

테스트 케이스	1	2	3	4
점수 범위	0~69점	70~79점	80~89점	90~100점
입력 값	60점	73점	86점	94점
예상 결과 값	0원	200만 원	400만 원	600만 원
실제 결과 값	0원	200만 원	400만 원	600만 원

명세 기반 테스트

- 경계 값 분석 기법
 - 경계에 있는 값을 테스트 데이터로 생성하여 테스트하는 방법
 - 경계 값과 경계 이전 값, 경계 이후 값을 가지고 테스트

표 9-7 경계 값 분석 기법의 테스트 예

테스트 케이스	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
입력 값	-1	0	1	69	70	71	79	80	81	89	90	91	99	100	101
예상 결과	오류	0	0	0	200	200	200	400	400	400	600	600	600	600	오류
실제 결과	오류	0	0	0	200	200	200	400	400	400	600	600	600	600	오류

명세 기반 테스트

- 원인-결과 그래프 기법

- 여러 입력 조건을 결합해 결과를 하나 이상 얻을 수 있으므로 이 단점을 극복할 수 있음
- 원인에 해당하는 입력 조건과 그 원인으로부터 발생하는 출력 결과를 가지고 만듦
- 이 그래프를 기초로 의사결정 테이블을 만들고 테스트 케이스를 위한 데이터는 이 의사결정 테이블을 이용해 작성

명세 기반 테스트

- 제작 과정

- ① 프로그램을 적합한 크기로 분할: 규모가 큰 프로그램은 원인-결과 그래프를 작성할 만한 크기로 분할
- ② 원인과 결과를 찾음: 요구분석명세서, 설계서, 프로그램에서 원인과 결과를 찾아 일련번호와 같은 식별자를 각각 부여
(원인: 하나의 입력 조건, 결과: 출력 조건)
- ③ 원인-결과 그래프 작성: 프로그램이 의미하는 내용을 분석해 이에 알맞은 원인과 결과를 연결하는 논리 그래프를 작성

표 9-8 원인-결과 그래프에서 사용되는 기호

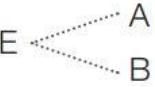
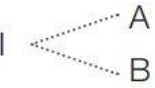
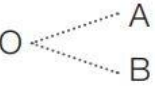
기호	표기법	사용 예	설명
동치(Equal, $\rightarrow$)	$A \rightarrow B$	if A then B	$A=1$ 이면 $B=1$, $A=0$ 이면 $B=0$
부정(NOT, $\sim$)	$A \sim B$	if not A then B	$A=1$ 이면 $B=0$, $A=0$ 이면 $B=1$
합(OR, $\vee$)	$A \vee B$	if A or B then C	(A, B, C)에서 $(1, 1)=1$, $(1, 0)=1$, $(0, 1)=1$, $(0, 0)=0$
곱(AND, $\wedge$)	$A \wedge B$	if A and B then C	(A, B, C)에서 $(1, 1)=1$, $(1, 0)=0$, $(0, 1)=0$, $(0, 0)=0$

명세 기반 테스트

- 제작 과정

- ④ 그래프에 제한 조건을 표시: 제한 조건 기호를 사용해 제한 조건을 그래프에 표시

표 9-9 원인-결과 그래프의 제한 조건 기호

명칭	기호	설명	
배타적exclusive 관계	E 	2개가 동시에 존재할 수 없다.	A=1이면 B=0 A=0이면 B=1 (A=B=0은 가능, A=B=1은 불가능)
포함inclusive 관계	I 	적어도 둘 중 한쪽은 성립한다.	A=1일 때 B=1 또는 0 B=1일 때 A=1 또는 0 (A=B=0은 불가능)
선택only one 관계	O 	항상 한쪽만 성립한다.	A=1이면 B=0 A=0이면 B=1 (A=B=0은 불가능)
필요require 관계	R 	한쪽이 성립하면 다른 쪽도 성립한다(B가 성립하기 위해 A가 반드시 필요하다).	A=1이면 B=1 또는 0 A=0이면 B=0 (B=1은 불가능)
강요mask 관계	M 	한쪽이 성립하면 다른 쪽은 성립하지 않는다.	A=1이면 B=0

명세 기반 테스트

- 제작 과정

- ⑤ 의사결정 테이블로 변환: 원인-결과 그래프를 바탕으로 결과로부터 그 결과가 생기는 조건을 추적해 그 사항을 기입한 의사결정 테이블로 변환

표 9-10 입력 항목에 대한 테스트 케이스

입력 항목 \ 테스트 케이스		1	2	3	4	5	6	7	8
총점	90 이상	T	T	F	F	F	F	F	F
	80 이상	F	F	T	T	F	F	F	F
	70 이상	F	F	F	F	T	T	F	F
	70 미만	F	F	F	F	F	F	T	T
리포트	제출	T	F	T	F	T	F	T	F

조합한 8개 항목에 대한 다음과 같은 예상 결과

1: 90점 이상(T), 리포트 제출(T) → A+
2: 90점 이상(T), 리포트 미제출(F) → A0
3: 80점 이상(T), 리포트 제출(T) → B+
4: 80점 이상(T), 리포트 미제출(F) → B0

5: 70점 이상(T), 리포트 제출(T) → C+
6: 70점 이상(T), 리포트 미제출(F) → C0
7: 70점 미만(T), 리포트 제출(T) → D
8: 70점 미만(T), 리포트 미제출(F) → F

명세 기반 테스트

- 제작 과정
 - ⑤ 의사결정 테이블로 변환

표 9-11 성적 처리에 대한 의사결정 테이블

입력 항목 \ 테스트 케이스		1	2	3	4	5	6	7	8
총점	90 이상	T	T	F	F	F	F	F	F
	80 이상	F	F	T	T	F	F	F	F
	70 이상	F	F	F	F	T	T	F	F
	70 미만	F	F	F	F	F	F	T	T
리포트	제출	T	F	T	F	T	F	T	F
결과 값	A ⁺	T	F	F	F	F	F	F	F
	A ⁰	F	T	F	F	F	F	F	F
	B ⁺	F	F	T	F	F	F	F	F
	B ⁰	F	F	F	T	F	F	F	F
	C ⁺	F	F	F	F	T	F	F	F
	C ⁰	F	F	F	F	F	T	F	F
	D	F	F	F	F	F	F	T	F
	F	F	F	F	F	F	F	F	T

- ⑥ 작성된 의사결정 테이블을 기반으로 테스트 케이스를 도출
(예) 테스트 케이스3번째의 테스트 데이터(총점 86점, 리포트 제출)를 생성했다면 결과 값은 B1

명세 기반 테스트

• 예제 9-1) 원인-결과 기법에 의한 테스트

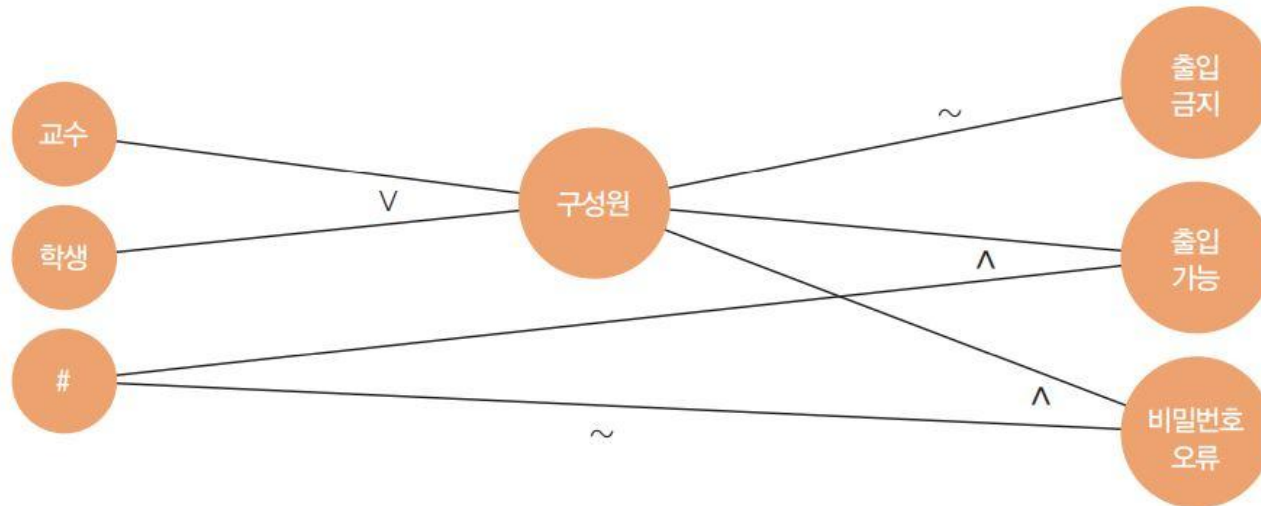
- 첫 번째 열이 P 또는 S로 시작하고, 두 번째 열이 #이면 '출입 가능'을 출력한다.
- 첫 번째 열이 P 또는 S로 시작하지 않으면 '출입 금지'를 출력한다.
- 첫 번째 열이 P 또는 S로 시작하고, 두 번째 열이 #이 아니면 '비밀번호 오류'를 출력한다.
- P는 교수(Professor), S는 학생(Student)을 의미한다.

① 원인과 결과를 찾는다.

원인	결과
원인 1: 첫 번째 열이 P	결과 1: '출입 가능' 출력
원인 2: 첫 번째 열이 S	결과 2: '출입 금지' 출력
원인 3: 두 번째 열이 #	결과 3: '비밀번호 오류' 출력

명세 기반 테스트

- 예제 9-1) 원인-결과 기법에 의한 테스트
 - ② 원인과 결과 그래프를 작성한다.



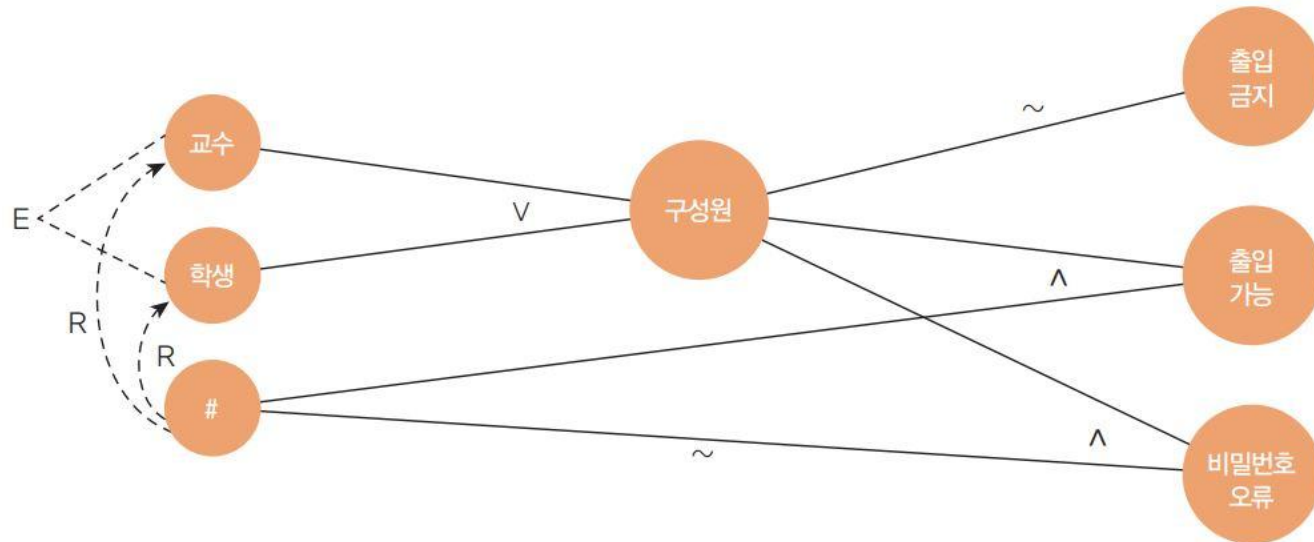
명세 기반 테스트

- 예제 9-1) 원인-결과 기법에 의한 테스트
 - ③ 원인-결과 그래프에 제한 조건을 표시한다.

입력 항목 \ 테스트 케이스		1	2	3	4	5
입력 값	원인 1(교수)	F	T	F	T	F
	원인 2(학생)	T	F	F	F	T
	원인 3(#)	T	T	X	F	F
중간 노드	구성원	T	T	F	T	T
출력 값	출입 금지	F	F	T	F	F
	출입 가능	T	T	F	F	F
	비밀번호 오류	F	F	F	T	T

명세 기반 테스트

- 예제 9-1) 원인-결과 기법에 의한 테스트
 - ④ 의사결정 테이블로 변환한다.



구현 기반 테스트

• 구현 기반 테스트의 개요

- 위장 내시경 검사, 조직 배양 검사, 개복해서 어느 부위에 어떤 이상이 생겼는지 확인하며 병을 진단
- 입력 데이터를 가지고 실행 상태를 추적함으로써 오류를 찾아내기 때문에 동적 테스트 부류에 속함
- 프로그램 내부에서 사용되는 변수나 서브루틴 등의 오류를 찾기 위해 프로그램 코드의 내부 구조를 테스트 설계의 기반으로 사용

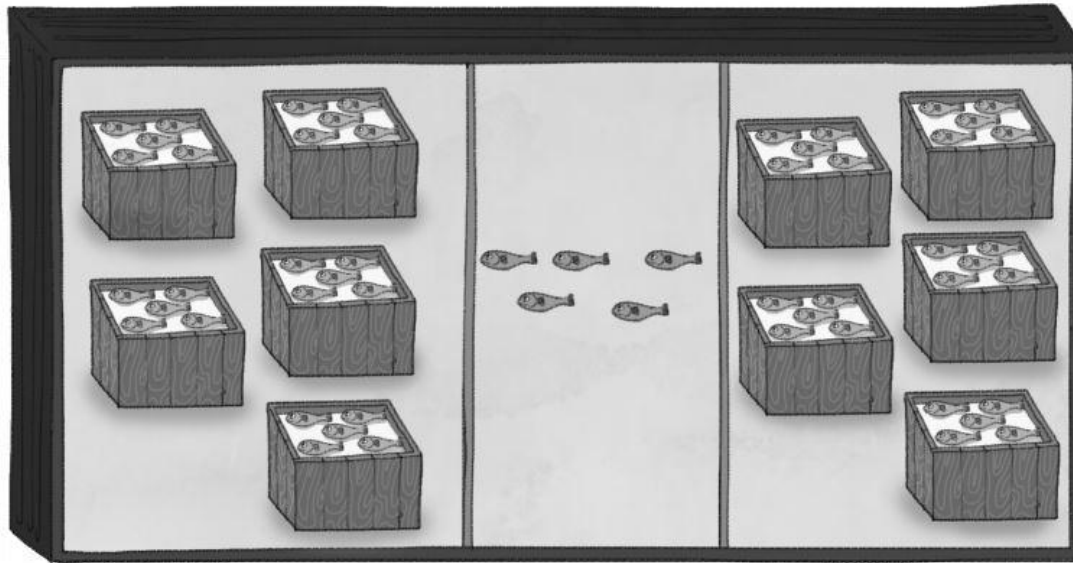


그림 9-15 구현 기반 테스트 비유: 위 내시경 검사

구현 기반 테스트

- 화이트박스 테스트 절차

- ① 테스트 데이터 적합성 기준을 선정한다.



- 컨테이너 차량 1대 = 대형 박스 10박스

- 대형 박스 1개 = 중형 박스 5개

- 중형 박스 1개 = 소형 박스 3개

- 소형 박스 1개 = 조기 10마리

그림 9-16 컨테이너 안의 조기 수량

구현 기반 테스트

- 화이트박스 테스트 절차

- ② 테스트 데이터를 생성한다.

- 테스트 데이터 적합성 기준을 근거로 한 가지 방법이 결정되면 명세나 원시 코드를 분석해 선정된 기준을 만족하는 입력 데이터를 만듦

- ③ 테스트를 실행한다. (테스트 방법)

- 문장 검증 기준
 - 분기 검증 기준
 - 조건 검증 기준
 - 분기/조건 검증 기준
 - 다중 조건 검증 기준
 - 기본 경로 테스트

구현 기반 테스트

- 문장 검증 기준

- ① 원시 코드를 제어 흐름 그래프 형태로 표현

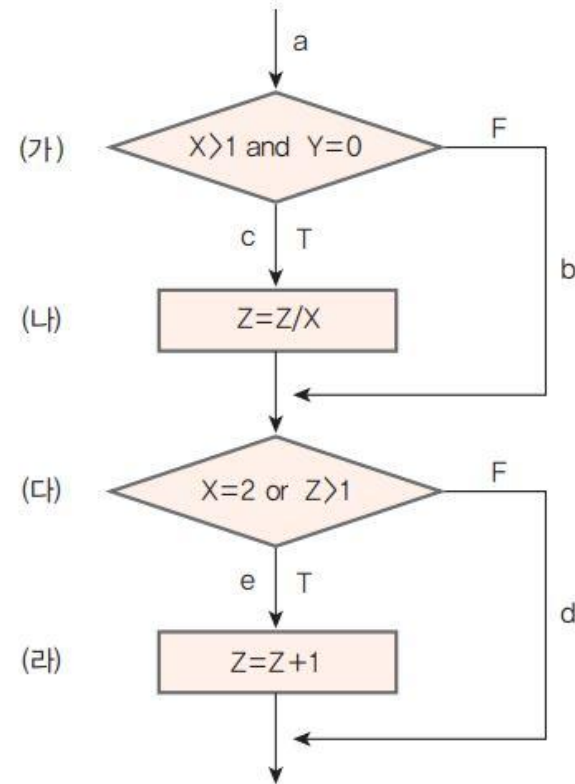


그림 9-17 제어 흐름으로 표현한 그래프

구현 기반 테스트

- 문장 검증 기준

- ① 원시 코드를 제어 흐름 그래프 형태로 표현

표 9-12 가능한 모든 경로

번호	경로(가, 다)	가능 경로	만족 여부	이유
1	경로 1 (T, T)	a-c-e	만족	(가), (나), (다), (라) 문장을 모두 지나감.
2	경로 2 (T, F)	a-c-d	불만족	(라) 문장을 안 지나감.
3	경로 3 (F, T)	a-b-e	불만족	(나) 문장을 안 지나감.
4	경로 4 (F, F)	a-b-d	불만족	(나), (라) 문장을 안 지나감.

- ② 모든 경로 중 문장 검증 기준을 만족하는 경로를 선택한다.

표 9-13 문장 검증 기준을 만족하는 경로

번호	경로(가, 다)	가능 경로	만족 여부	이유
1	경로 1 (T, T)	a-c-e	만족	(가), (나), (다), (라) 문장을 모두 지나감.

구현 기반 테스트

- 문장 검증 기준

③ 선택한 경로에 해당하는 테스트 데이터를 가지고 실행한다.

표 9-14 테스트 데이터로 실행

번호	경로(가, 다)	테스트 데이터	가능 경로	출력 값	만족 여부
1	경로 1 (T, T)	X=2, Y=0, Z=3	a-c-e	2.5	만족

구현 기반 테스트

- 문장 검증 기준

- 문장 검증 기준의 문제점

and인 경우: 입력 값($X=2, Y=0, Z=3$), 출력 값($Z=2.5$)

or인 경우: 입력 값($X=2, Y=0, Z=3$), 출력 값($Z=2.5$)

- 첫 번째 조건문에서 원래는 or인데 실수로 and로 코딩했다면 이 문장 검증 기준 방법으로는 오류를 발견하지 못함
 - and인 경우에 (나) 문장을 지나가고 or인 경우에도 (나) 문장을 지나가기 때문
 - 두 경우 모두 결과가 같아 원래 or인 경우를 and로 실수해도 오류를 찾지 못함
 - 두 번째 조건문에서 $Z>1$ 식을 $Z>0$ 로 잘못 코딩해도 오류를 발견하지 못함
 - 연산자 or의 특성이 둘 중 하나만 만족하면 다른 조건식은 결과에 영향을 주지 않기 때문
 - 문장 검증 기준은 다른 검증 기준에 비해 약하며 최소한의 테스트

구현 기반 테스트

• 분기 기반 검증(결정 검증 기준)

- 테스트 케이스를 선정하는 기준은 원시 코드에 존재하는 조건문에 대해 T가 되는 경우와 F가 되는 경우가 최소한 한 번은 실행되는 입력 데이터를 테스트 케이스로 사용하는 것
- 사용시 분기 시점에서 조건과 관련된 오류를 찾을 수 있고, 또 분기가 합류하는 위치에서 오류를 발견할 가능성이 높음

① 원시 코드를 제어 흐름 그래프 형태로 표현

- 원시 코드를 제어 흐름 구조의 그래프 형태로 바꾸어 표현

② 가능한 모든 경로를 구함

- 가능한 모든 경로를 나타내고, 분기 검증 기준의 만족 여부를 나타냄

표 9-15 가능한 모든 경로

번호	경로(가, 다)	가능 경로	만족 여부	이유
1	경로 1(T, T)	a-c-e	불만족	(F, F) 경로를 테스트 안 함.
2	경로 2(T, F)	a-c-d	불만족	(F, T) 경로를 테스트 안 함.
3	경로 3(F, T)	a-b-e	불만족	(T, F) 경로를 테스트 안 함.
4	경로 4(F, F)	a-b-d	불만족	(T, T) 경로를 테스트 안 함.

구현 기반 테스트

- 분기 기반 검증(결정 검증 기준)

- ③ 모든 경로 중 분기 검증 기준을 만족하는 경로를 선택

- 네 개의 경로 중 하나만으로는 분기 검증 기준을 만족시키지 못함

- 두 개의 경로를 묶어서 분기 검증 기준을 만족시킬 수 있는 경우를 찾음

- $(1, 2), (1, 3), (1, 4), (2, 3), (2, 4), (3, 4)$ $(1, 4)$ 또는 $(2, 3)$

구현 기반 테스트

• 분기 기반 검증(결정 검증 기준)

표 9-16 1과 4의 테스트 케이스

번호	경로(가, 다)	가능 경로	이유
1	경로 1 (T, T)	a-c-e	(T, T) 경로를 테스트함.
4	경로 4 (F, F)	a-b-d	(F, F) 경로를 테스트함.

표 9-17 2와 3의 테스트 케이스

번호	경로(가, 다)	가능 경로	이유
2	경로 2 (T, F)	a-c-d	(T, F) 경로를 테스트함.
3	경로 3 (F, T)	a-b-e	(F, T) 경로를 테스트함.

표 9-18 1, 4의 테스트 결과

번호	경로(가, 다)	테스트 데이터	가능 경로	출력 값	두 경로의 합
1	경로 1 (T, T)	X=2, Y=0, Z=10	a-c-e	Z=6	만족
4	경로 4 (F, F)	X=0, Y=0, Z=1	a-b-d	Z=1	

표 9-19 2, 3의 테스트 결과

번호	경로(가, 다)	테스트 데이터	가능 경로	출력 값	두 경로의 합
2	경로 2 (T, F)	X=3, Y=0, Z=2	a-c-d	Z=1	만족
3	경로 3 (F, T)	X=2, Y=1, Z=2	a-b-e	Z=3	

구현 기반 테스트

- 분기 기반 검증(결정 검증 기준)
 - 분기 기반 검증의 한계
 - 경로 1, 4: $Z > 1$ 를 $Z < 1$ 로 코딩 실수를 해도 그 오류를 발견 못함
 - 개별 조건식이 or로 연결되어 있어 둘 중 하나만 만족하면 두 번째 식이 뭐든 조건문의 결과 값에 영향을 주지 않기 때문
 - 조건문 내의 개별 조건식에 대해 각각 T와 F인 경우를 최소한 한 번씩 수행할 수 있는 테스트 케이스를 선정해 해결

구현 기반 테스트

- 조건 검증 기준

- 조건문(전체 조건식)이 IF(score >= 90 and report >= 90) THEN printf("A")일 경우
 - 두 개별 조건식이 T일 때만 조건문이 T이고, 나머지는 두 조건식과 관계없이 모두 F
 - 두 개별 조건식이 and로 연결되어 있어 개별 조건식이 (T, T)를 제외하고는 나머지 (T, F), (F, T), (F, F)는 모두 거짓이 됨

표 9-20 and 연산자 사용 시 조건문의 결과

개별 조건식 A (score >= 90)	연산자	개별 조건식 B (report >= 90)	조건문 (전체 조건식)
T	and	T	T
T		F	F
F		T	F
F		F	F

구현 기반 테스트

- 조건 검증 기준

- 조건문(전체 조건식)이 IF(score>=90 and report>=90) THEN printf("A")일 경우
 - 개별 조건식 A(score>=90)가 T이고, B(report >=90)가 F인 경우

- 개별 조건식 A: T에서 F로 바뀌는 경우
- (F, T)에서 개별 조건식 B: T에서 F로 바뀌는 경우 모두 오류 발견 못함
- (F, F)에서 개별 조건식 둘 중 하나가 T로 바뀌는 경우

- 연산자 and의 특성이 하나라도 F이면 전체 조건식이 항상 F이기 때문
- 두 개의 개별 조건식이 존재할 때 개별 조건식의 T와 F를 최소한 한 번은 테스트할 수 있도록 테스트 케이스 선정
- 분기 검증 기준에서 발견하지 못한 오류(개별 조건식에 존재하는 오류)를 발견할 수 있는 더 강력한 테스트가 됨

구현 기반 테스트

- 조건 검증 기준

- 조건 검증 기준의 4가지 테스트 케이스

(다) : $(X=2) = T \text{ or } (Z>1) = T$
(다) : $(X=2) = T \text{ or } (Z>1) = F$
(다) : $(X=2) = F \text{ or } (Z>1) = T$
(다) : $(X=2) = F \text{ or } (Z>1) = F$

(다)의 개별 테스트 조건식을 만족하는 케이스

(T, T), (F, F)

(T, F), (F, T)

(가): $(X>1)=T \text{ and } (Y=0)=T$
(가): $(X>1)=T \text{ and } (Y=0)=F$
(가): $(X>1)=F \text{ and } (Y=0)=T$
(가): $(X>1)=F \text{ and } (Y=0)=F$

(가)의 개별 테스트 조건식을 만족하는 케이스

(T, T), (F, F)

(T, F), (F, T)

구현 기반 테스트

- 조건 검증 기준
 - 조건 검증 기준의 4가지 테스트 케이스

표 9-21 [그림 9-17]에 대한 테스트 케이스

1

(가) 개별 조건식		(다) 개별 조건식	
A	B	A	B
T	T	T	T
F	F	F	F

2

(가) 개별 조건식		(다) 개별 조건식	
A	B	A	B
T	T	T	F
F	F	F	T

3

(가) 개별 조건식		(다) 개별 조건식	
A	B	A	B
T	T	F	T
F	F	T	F

4

(가) 개별 조건식		(다) 개별 조건식	
A	B	A	B
T	T	F	F
F	F	T	T

5

(가) 개별 조건식		(다) 개별 조건식	
A	B	A	B
T	F	T	T
F	T	F	F

6

(가) 개별 조건식		(다) 개별 조건식	
A	B	A	B
T	F	T	F
F	T	F	T

7

(가) 개별 조건식		(다) 개별 조건식	
A	B	A	B
T	F	F	T
F	T	T	F

8

(가) 개별 조건식		(다) 개별 조건식	
A	B	A	B
T	F	F	F
F	T	T	T

구현 기반 테스트

- 조건 검증 기준
 - 조건 검증 기준의 4가지 테스트 케이스

표 9-22 [그림 9-17]에 대한 테스트 케이스

번호	경로			개별 조건식	두 경로의 합	테스트 케이스	전체 조건식	
	(가)	(다)						
1	경로 1	T, T	T, T	불만족	만족	적합	(가)T, (다)T	만족
	경로 2	F, F	F, F	불만족			(가)F, (다)F	
2	경로 3	T, T	T, F	불만족	만족	적합	(가)T, (다)T	불만족/ (다)F가 없음
	경로 4	F, F	F, T	불만족			(가)F, (다)T	
3	경로 5	T, T	F, T	불만족	만족	제외(경로 3, 4와 중복)	(가)T, (다)T	X
	경로 6	F, F	T, F	불만족			(가)F, (다)T	
4	경로 7	T, T	F, F	불만족	만족	제외(경로 1, 2와 중복)	(가)T, (다)F	X
	경로 8	F, F	T, T	불만족			(가)F, (다)T	
5	경로 9	T, F	T, T	불만족	만족	적합	(가)F, (다)T	불만족/ (가)T가 없음
	경로 10	F, T	F, F	불만족			(가)F, (다)F	
6	경로 11	T, F	T, F	불만족	만족	적합	(가)F, (다)T	불만족/(가)T, (다)F가 없음
	경로 12	F, T	F, T	불만족			(가)F, (다)T	
7	경로 13	T, F	F, T	불만족	만족	제외(경로 11, 12와 중복)	(가)F, (다)T	X
	경로 14	F, T	T, F	불만족			(가)F, (다)T	
8	경로 15	T, F	F, F	불만족	만족	제외(경로 9, 10과 중복)	(가)F, (다)F	X
	경로 16	F, T	T, T	불만족			(가)F, (다)T	

구현 기반 테스트

- 분기/조건 검증 기준

- 분기/조건 검증 기준: 개별 조건식을 모두 만족하면서 전체 조건식도 만족하는 테스트 케이스

표 9-23 분기/조건 검증 기준의 테스트 케이스

번호	경로			개별 조건식	두 경로의 합	전체 조건식	
	(가)	(다)					
1	경로 1	T, T	T, T	불만족	만족	(가)T, (다)T	만족
	경로 2	F, F	F, F	불만족		(가)F, (다)F	

- 예시의 개별 조건식($Y=0$)에서 오류가 발생해도 이를 발견할 수 없음
- 2개의 개별식이 and로 연결되었기 때문에 개별 조건식 하나만 F이면 조건문은 무조건 F이기 때문
- 마스크: 어떤 개별 조건식이 다른 개별 조 건식의 결과와 상관없이 이미 결정되는 것

구현 기반 테스트

- 다중 조건 검증 기준

- 마스크 문제 해결 방법: and로 연결된 개별 조건식

- and로 연결된 개별 조건식은 두 식 중 하나가 F인 경우 나머지 식이 F이든 T이든 상관없이 결과가 F
 - 이를 해결하기 위해 나머지 식은 T인 경우와 F인 경우를 각각 하나씩 추가해 이를 만족하는 테스트 케이스를 만들어 테스트

- 마스크 문제 해결 방법: or로 연결된 개별 조건식

- or로 연결된 개별 조건식은 두 식 중 하나가 T인 경우 나머지 식은 F이든 T이든 상관없이 결과가 T
 - 이를 해결하기 위해 나머지 식은 T인 경우와 F인 경우를 하나씩 추가해 이를 만족하는 테스트 케이스를 만들어 테스트
 - 다중 조건 검증 기준: T마스크 문제까지 해결한 테스트 케이스에 해당하는 테스트 데이터를 생성하는 기준

구현 기반 테스트

- 다중 조건 검증 기준

표 9-24 다중 조건 검증 기준의 테스트 케이스

번호	경로		분기/조건 검증 기준		다중 조건 검증 기준
	(가)	(다)	두 경로의 합	전체 조건식	
1	경로 1	T, T	만족	만족	만족
	경로 1	T, F			
	경로 2	F, F			
	경로 2	T, F			

경로 1: or가 T인 경우에 해당

경로 2: and가 F인 경우에 해당

구현 기반 테스트

- 기본 경로 테스트

- 기본 경로 테스트는 매케이브가 만듦
- 원시 코드의 독립적인 경로가 최소한 한 번은 실행되는 테스트 케이스를 찾아 테스트를 수행
- 원시 코드의 독립적인 경로를 모두 수행하는 것을 목적으로 함
 - ① 순서도 작성

구현 기반 테스트

- 기본 경로 테스트

- ① 순서도 작성

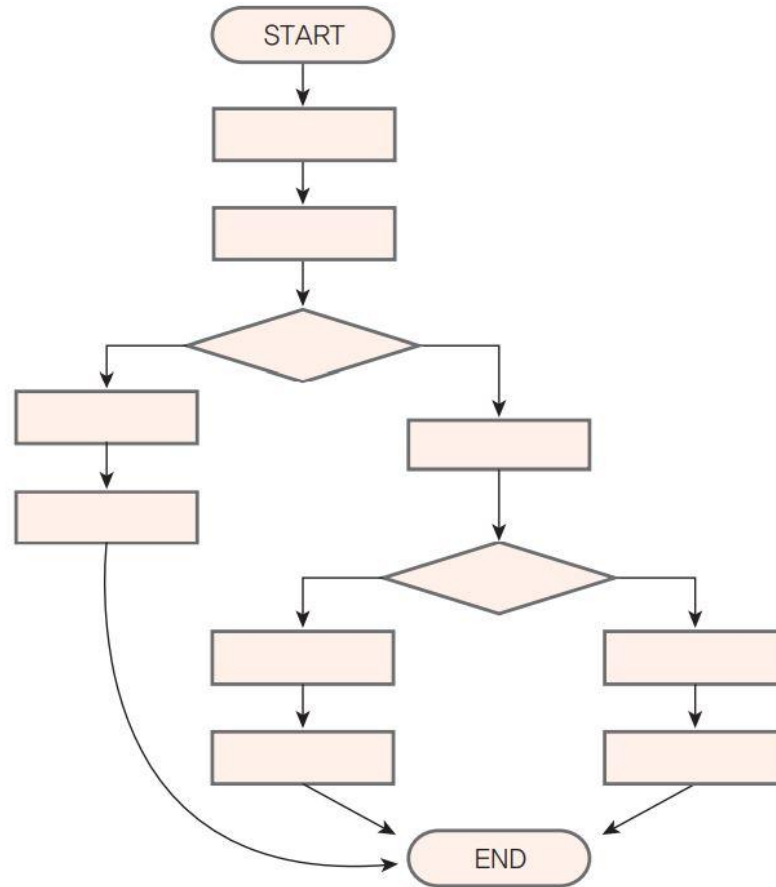


그림 9-19 순서도의 표현

구현 기반 테스트

- 기본 경로 테스트

- ② 흐름 그래프를 작성

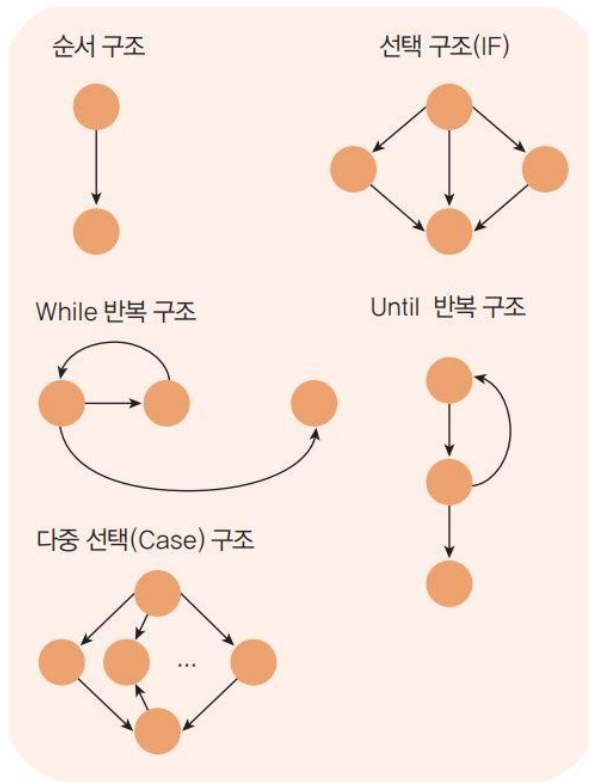


그림 9-20 흐름 그래프에서 사용되는 표기(R^{Region} : 영역)

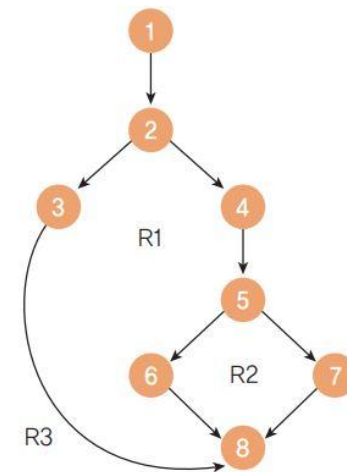
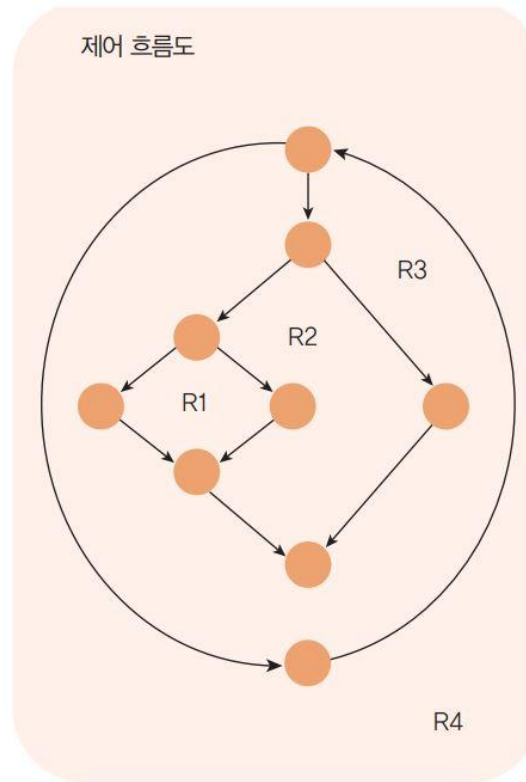


그림 9-21 흐름 그래프의 표현(R^{Region} : 영역)

구현 기반 테스트

• 기본 경로 테스트

③ 순환 복잡도(CC)를 계산

- 순환 복잡도: 매케이브가 정의한 메트릭으로 원시 코드의 복잡도를 정량적으로 평가하는 방법
- 순환 복잡도 계산 공식

$$\begin{aligned}CC &= R \text{의 수} = 3 \\ CC &= E - N + 2 = 9 - 8 + 2 = 3 \\ CC &= P + 1 = 2 + 1 = 3\end{aligned}$$

[순환 복잡도 공식]

- CC5R의 수
- CC5E2N12
- CC5P11
 - R(Region) : 화살표와 노드로 둘러싸인 구역
외부 구역도 하나의 영역으로 간주함
 - E(Edge) : 화살표 수
 - N(Node) : 노드 수
 - P(Predicate) : 분기 노드 수

구현 기반 테스트

- 기본 경로 테스트

- ③ 독립적인 경로를 정의

- 경로 1: 1-2-3-8
 - 경로 2: 1-2-4-5-6-8
 - 경로 3: 1-2-4-5-7-8

- ④ 테스트 케이스를 작성

소프트웨어 개발 단계에 따른 테스트

V 모델

- V 모델의 개요
 - 요구사항 정의, 분석, 설계, 구현 단계와 테스트 과정의 단위 테스트, 통합 테스트, 시스템 테스트, 인수 테스트가 V자로 연결
- V 모델 과정에 따른 테스트
 - 단위 테스트, 통합 테스트, 시스템 테스트, 인수 테스트, 회귀 테스트

소프트웨어 개발 단계에 따른 테스트



소프트웨어 개발 단계에 따른 테스트

V 모델

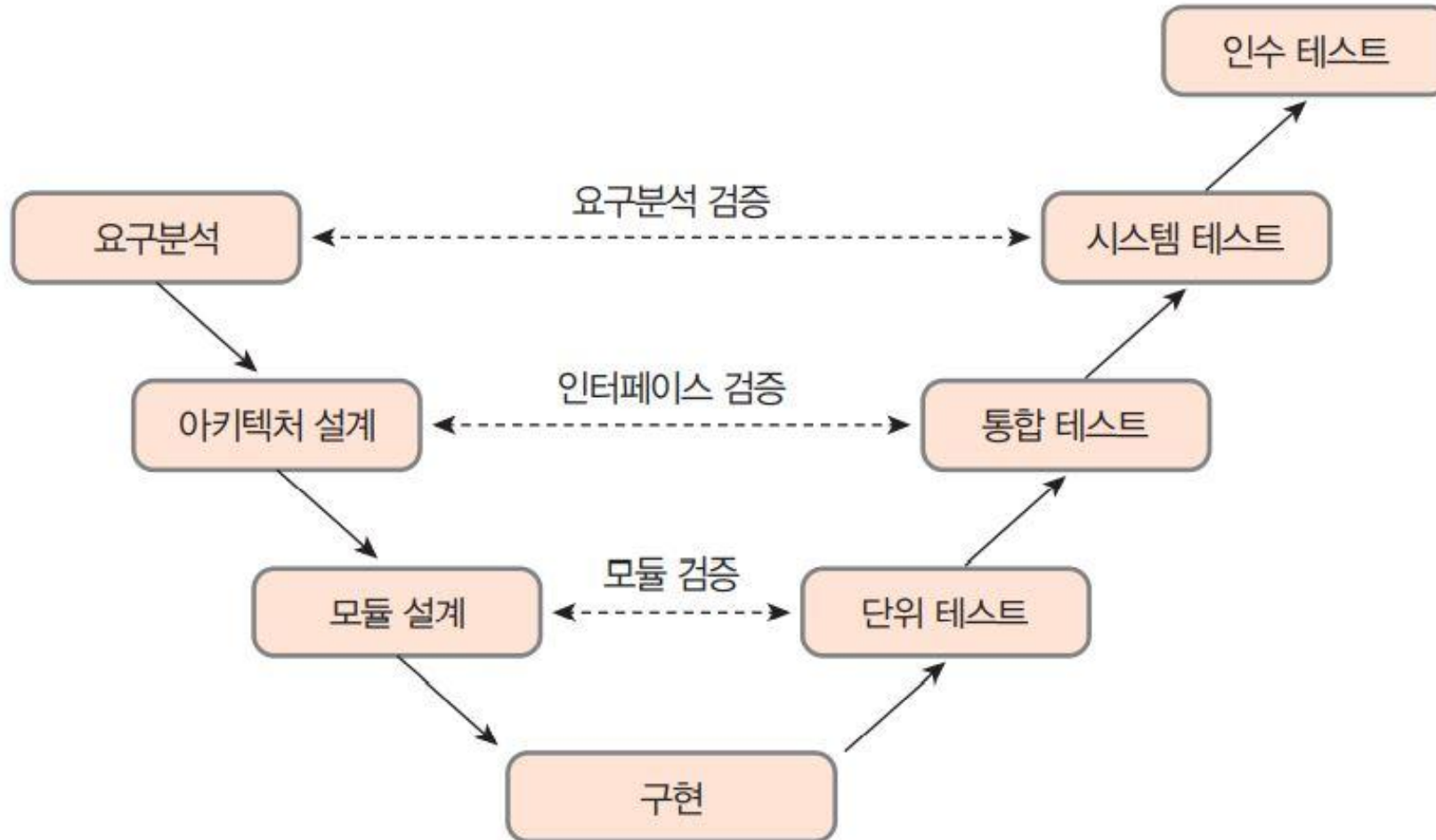


그림 9-22 V 모델

소프트웨어 개발 단계에 따른 테스트

단위 테스트

- 단위 테스트(모듈 테스트)

- 프로그램의 기본 단위인 모듈을 테스트
- 구현 단계에서 각 모듈의 개발을 완료한 후 개발자가 요구분석명세서대로 정확히 구현되었는지 테스트
- 개별 모듈이 제대로 구현되어 정해진 기능을 정확히 수행하는지를 테스트
- 한 모듈을 테스트하려면 그 모듈과 직접 관련된 상위 모듈과 하위 모듈까지 모두 존재해야 함

- 드라이버와 스텝

- 테스트 드라이버: 상위 모듈의 역할을 하는 가상의 모듈, 테스트할 모듈을 호출하는 역할
- 스텝: 하위 모듈의 역할을 하는 가상의 모듈, 테스트할 모듈이 호출할 때 인자를 통해 받은 값을 가지고 수행한 후 그 결과를 테스트할 모듈에 넘겨주는 역할
- 드라이버와 스텝 모듈은 테스트할 때 필요한 기능만 제공할 수 있도록 단순하게 구현

소프트웨어 개발 단계에 따른 테스트

단위 테스트

- 단위 테스트 수행 후 발견되는 오류
 - 잘못 사용한 자료형
 - 잘못된 논리 연산자
 - 알고리즘 오류에 따른 원치 않는 결과
 - 틀린 계산 수식에 의한 잘못된 결과
 - 탈출구가 없는 반복문의 사용



그림 9-23 드라이버/스텝 모듈과 테스트 대상 모듈의 관계

소프트웨어 개발 단계에 따른 테스트

통합 테스트

- **통합 테스트의 개요**
 - 단위 테스트가 끝난 모듈을 통합하는 과정에서 발생할 수 있는 오류를 찾는 테스트
 - '모듈 간의 상호작용이 정상적으로 수행되는가' 테스트
 - 모듈 사이의 인터페이스 오류는 없는지, 모듈이 올바르게 연계되어 동작하고 있는지 체크
- **모듈 통합 방법에 따른 분류**
 - 한꺼번에 하는 방법: big-bang 테스트
 - 점진적으로 하는 방법: 하향식 기법, 상향식 기법

소프트웨어 개발 단계에 따른 테스트

통합 테스트

- Big-bang 테스트
 - 단위 테스트가 끝난 모듈을 한꺼번에 결합해 테스트를 수행하는 방식
 - 이 방법은 소규모 프로그램이나 프로그램의 일부를 대상으로 하는 경우가 많고 그만큼 절차가 간단하고 쉬움
 - 한꺼번에 통합하면 오류가 발생했을 때 어떤 모듈에서 오류가 존재하고 또 그 원인이 무엇인지 찾기가 매우 어려움

소프트웨어 개발 단계에 따른 테스트

통합 테스트

- 점진적 모듈 통합 방법: 하향식 기법

- 시스템을 구성하는 모듈의 계층 구조에서 맨 위의 모듈부터 시작해 점차 하위 모듈 방향으로 통합하는 방법
- 최상위 모듈 A를 모듈 B와 통합해 테스트를 실시
- 그 다음으로 모듈 C를 먼저 선택할지(넓이 우선 방식), 아니면 B 아래에 있는 모듈 E를 먼저 선택할지(깊이 우선 방식) 결정
 - 넓이 우선 방식: $(A, B) \rightarrow (A, C) \rightarrow (A, D) \rightarrow (A, B, E) \rightarrow (A, B, F) \rightarrow (A, C, G)$ 와 같이 같은 행에서 는 옆으로 가며 통합 테스트
 - 깊이 우선 방식: $(A, B) \rightarrow (A, B, E) \rightarrow (A, B, F) \rightarrow (A, C) \rightarrow (A, C, G) \rightarrow (A, D)$ 순으로 통합하며 테스트
- 하향식 방식에서는 상위 모듈부터 테스트를 수행하기 때문에 단위 테스트에서 설명한 스텝 모듈이 필요

소프트웨어 개발 단계에 따른 테스트

통합 테스트

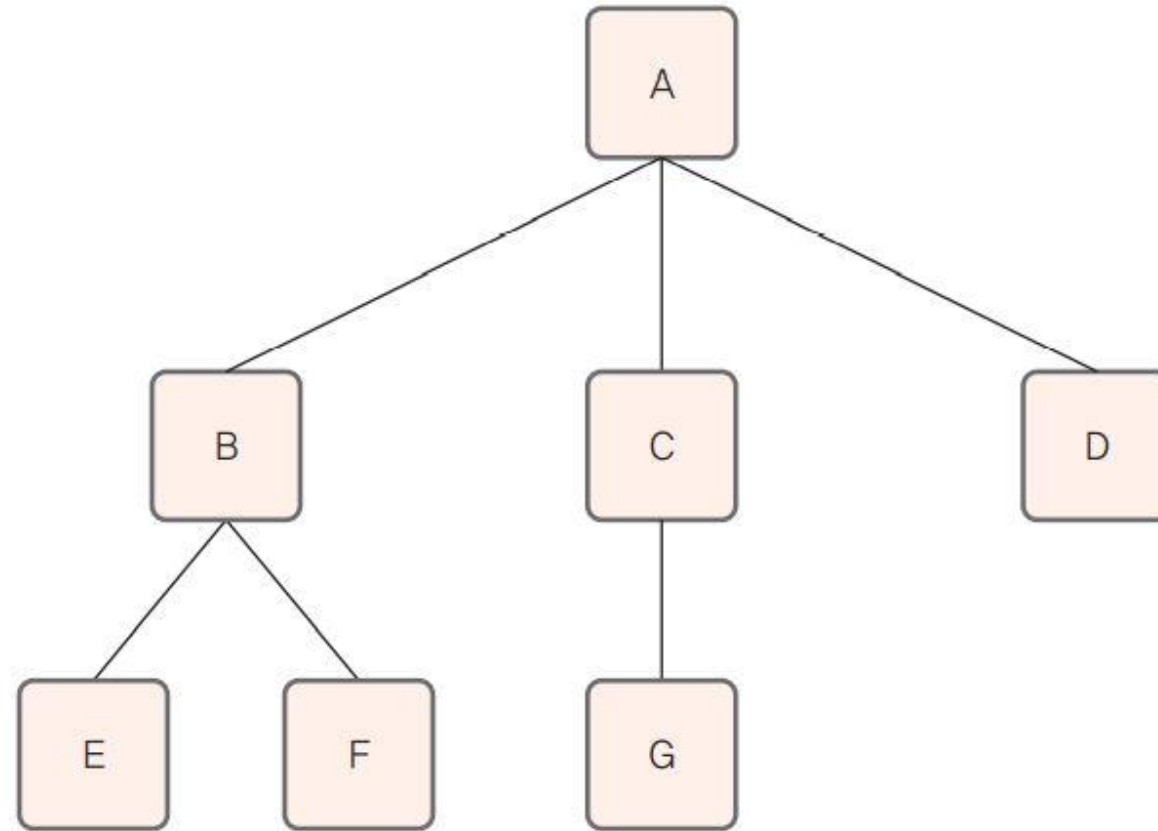


그림 9-24 점진적 모듈 통합 방법을 설명하기 위한 모듈 구성도

소프트웨어 개발 단계에 따른 테스트

통합 테스트

- 점진적 모듈 통합 방법: 상향식 기법

- 하향식 기법과는 반대로 가장 아래에 있는 모듈부터 테스트를 시작
- 하향식에서 스텝이 필요했다면, 상향식에서는 상위 모듈의 역할을 하는 테스트 드라이버가 필요
 - 드라이버는 하위 모듈을 순서에 맞게 호출하고, 호출할 때 필요한 매개변수를 제공하며, 반환값을 전달하는 역할

- 상향식 기법의 순서

- ① 모듈 E와 F를 모듈 B에 통합해 테스트
- ② 모듈 G를 모듈 C에 통합해 테스트
- ③ 모듈 B, C, D를 모듈 A에 통합해 테스트

소프트웨어 개발 단계에 따른 테스트

통합 테스트

- 점진적 모듈 통합 방법: 상향식 기법
 - 상향식 기법의 장점과 단점
 - 최하위 모듈들을 개별적으로 병행해 테스트할 수 있기 때문에 하위에 있는 모듈들을 충분히 테스트할 수 있다는 점
 - 정밀한 계산이나 데이터 처리가 요구되는 시스템 같은 경우에 사용하면 좋음
 - 상위 모듈에 오류가 발견되면 그 모듈과 관련된 하위의 모듈을 다시 테스트해야 하는 번거로움이 생김

소프트웨어 개발 단계에 따른 테스트

통합 테스트

- 시스템 테스트
 - 시스템 테스트의 개요
 - 시스템 전체가 정상적으로 작동하는지를 체크
 - 모듈이 모두 통합된 후 사용자의 요구 사항들을 만족하는지 테스트
 - 사용자에게 개발된 시스템을 전달하기 전에 개발자가 진행하는 마지막 테스트
 - V & V에서 확인에 해당
 - 실제 사용 환경과 유사하게 테스트 환경을 만들고 요구 분석 명세서에 명시한 기능적, 비기능적 요구 사항을 충족하는지 테스트
 - 주로 부하를 주는 상황에서 수행하고, 비기능적 테스트를 중심으로 수행

소프트웨어 개발 단계에 따른 테스트

통합 테스트

- 인수 테스트
 - 인수 테스트의 개요
 - 시스템이 예상대로 동작하는지 확인하고, 요구사항에 맞는지 확신하기 위해 하는 테스트
 - 시스템을 인수하기 전 요구분석명세서에 명시된 대로 모두 충족시키는지 사용자가 테스트
 - 인수 테스트가 끝나면 사용자는 인수를 승낙한 것
 - 시스템이 사용자에게 정상적으로 인수되면 프로젝트는 종료
 - 인수 테스트 시 인수 테스트 계획서는 사용자가 직접 작성하거나, 개발자가 작성한 것을 사용자가 승인

소프트웨어 개발 단계에 따른 테스트

통합 테스트

- 인수 테스트

- 알파 테스트(내부 필드 테스트)

- 개발 완료된 소프트웨어를 베타 테스트 전에 회사 내의 다른 직원에게 개발자 환경에서 테스트
 - 개발자가 그들이 사용하는 것을 보면서 오류와 사용상의 문제점을 파악

- 베타 테스트

- 개발 완료되어 알파 테스트를 거친 소프트웨어를 시장에 상품으로 내놓기 전에 시장의 피드백을 얻기 위한 목적으로 수행
 - 특정 사용자에게 미리 사용해 보도록 배포
 - 베타 버전을 받은 사용자들이 사용자 입장에서 사용해보고 문제점이나 오류를 개발자에게 알려주는 방식으로 테스트
 - 그런 다음 개발자가 베타 테스트를 통해 보고된 문제점을 수정해 제품을 출시

소프트웨어 개발 단계에 따른 테스트

회귀 테스트

- **확정 테스트**
 - 원시 코드의 결함을 수정한 후 제대로 수정되었는지 확인하는 테스트
- **회귀 테스트**
 - 한 모듈의 수정이 다른 부분에 영향을 끼칠 수도 있다고 생각하여 수정된 모듈뿐 아니라 관련된 모듈까지 문제가 없는지 테스트
 - 한 모듈의 수정이 다른 부분에 미치는 영향을 최소화하기 위해 필요

소프트웨어 개발 단계에 따른 테스트

회귀 테스트

① 수정을 위한 회귀 테스트

- 모든 테스트를 완료하여 사용자에게 전달하기 전에 테스트 과정에서 미처 발견하지 못한 오류를 찾아 수정한 후 다시 테스트

② 점진적 회귀 테스트

- 사용 중에 일부 기능을 추가하여 새로운 버전을 만들고, 이 새 버전을 다시 테스트
- 테스트 케이스의 일부분을 재실행할 수도 있고, capture/playback 도구를 이용해 자동으로 수행할 수 있음

END

